

Teach Module 22: Spring Core and Inversion of Control

This note is based on the Module 22 topic list from `docs/Java Software Engineer bootcamp.docx` , but the teaching content below is original explanatory material and does not reuse the course material.

Module 22 Overview

Module 22 covers **Spring Core and Inversion of Control**.

The main idea is simple:

In Spring, your classes declare what they need, and the Spring container provides it.

In plain Java, if one class needs another, you often create the dependency yourself:

```
UserService service = new UserService(new UserRepository());
```

That means your code controls object creation.

In Spring, you usually let the framework create and connect objects:

```
@Service
public class UserService {
    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
}
```

Spring sees that `UserService` needs a `UserRepository` , finds or creates one, and injects it.

That is the core of **Inversion of Control**, also called **IoC**.

1. Beans

A **bean** is an object managed by Spring.

If Spring creates it, stores it, wires it, and manages its lifecycle, it is a bean.

Common ways to define beans:

```
@Component
public class EmailService {
}
```

Or:

```
@Configuration
public class AppConfig {
```

```
@Bean
public EmailService emailService() {
    return new EmailService();
}
```

Use `@Component` when Spring can directly create the class.

Use `@Bean` when you need custom construction logic, or when the class comes from a library and you cannot annotate it.

2. Application Context

The **ApplicationContext** is Spring's container.

It knows:

- which beans exist
- how to create them
- how they depend on each other
- when to initialize and destroy them

In a Spring Boot application, the context starts here:

```
SpringApplication.run(MyApplication.class, args);
```

That line starts the Spring container.

3. Dependency Injection

Dependency Injection means an object receives the things it needs instead of creating them itself.

Constructor Injection

Constructor injection is the preferred style:

```
@Service
public class OrderService {
    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Constructor injection is best because dependencies are required, immutable, visible, and easy to test.

Setter Injection

```
@Service
public class OrderService {
```

```

    private PaymentService paymentService;

    @Autowired
    public void setPaymentService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}

```

Setter injection can be useful for optional dependencies.

Field Injection

```

@Service
public class OrderService {
    @Autowired
    private PaymentService paymentService;
}

```

Field injection works, but it hides dependencies and makes testing harder.

Rule of thumb:

Use constructor injection by default.

4. Component Scanning

Spring finds classes by scanning packages for annotations such as:

- `@Component`
- `@Service`
- `@Repository`
- `@Controller`
- `@RestController`
- `@Configuration`

These are called stereotype annotations.

They all mark a class as a Spring-managed component, but they communicate different intent:

- `@Component` : generic Spring bean
- `@Service` : business logic
- `@Repository` : data access
- `@Controller` : web MVC controller
- `@RestController` : REST API controller
- `@Configuration` : configuration class containing bean definitions

Example:

```

@Repository
public class UserRepository {
}

```

```

@Service
public class UserService {
    private final UserRepository repository;

    public UserService(UserRepository repository) {
        this.repository = repository;
    }
}

```

Spring discovers both, creates both, and injects `UserRepository` into `UserService` .

5. Loose Coupling

IoC helps you avoid tightly coupled code.

Tightly coupled code:

```

public class ReportService {
    private PdfExporter exporter = new PdfExporter();
}

```

`ReportService` is locked to `PdfExporter` .

Loosely coupled code:

```

public interface Exporter {
    void export(String content);
}

```

```

@Component
public class PdfExporter implements Exporter {
    public void export(String content) {
        System.out.println("Exporting PDF");
    }
}

```

```

@Service
public class ReportService {
    private final Exporter exporter;

    public ReportService(Exporter exporter) {
        this.exporter = exporter;
    }
}

```

Now `ReportService` depends on an abstraction instead of one concrete class.

That makes the code easier to test, swap, and extend.

6. Bean Lifecycle

A Spring bean roughly goes through this journey:

1. Spring creates the object.
2. Spring injects dependencies.
3. Spring runs initialization callbacks.
4. The bean is used by the application.
5. Spring destroys the bean when the context shuts down.

You can hook into lifecycle events:

```
@PostConstruct
public void init() {
    System.out.println("Bean initialized");
}

@PreDestroy
public void cleanup() {
    System.out.println("Bean destroyed");
}
```

Use lifecycle methods sparingly. Most business logic should not live in lifecycle methods.

Mini Example

```
public interface NotificationService {
    void send(String message);
}
```

```
@Service
public class EmailNotificationService implements NotificationService {
    public void send(String message) {
        System.out.println("Email sent: " + message);
    }
}
```

```
@Service
public class AccountService {
    private final NotificationService notificationService;

    public AccountService(NotificationService notificationService) {
        this.notificationService = notificationService;
    }

    public void createAccount(String username) {
        System.out.println("Account created for " + username);
        notificationService.send("Welcome, " + username);
    }
}
```

```
}  
}
```

`AccountService` does not create `EmailNotificationService` .

It simply says: "I need a `NotificationService` ."

Spring handles the rest.

That is IoC in action.

Practice Exercises

Exercise 1: Create Basic Beans

Build a small Spring Boot app with these classes:

- `GreetingService`
- `TimeService`
- `MessagePrinter`

Make each one a Spring bean using `@Component` or `@Service` .

Goal: confirm Spring creates and manages these objects.

Example:

```
@Component  
public class GreetingService {  
    public String greet(String name) {  
        return "Hello, " + name;  
    }  
}
```

Then inject it into another class.

Exercise 2: Constructor Injection

Create:

- `UserService`
- `UserRepository`

`UserService` should depend on `UserRepository` .

Use constructor injection only.

Example task:

```
userService.registerUser("Asha");
```

Expected behavior:

```
Saving user: Asha
```

Goal: understand how Spring injects dependencies.

Exercise 3: Compare Injection Styles

Create three versions of the same service:

- `ConstructorInjectedService`
- `SetterInjectedService`
- `FieldInjectedService`

Each should use the same dependency, such as `NotificationService` .

Then compare:

- Which one is easiest to test?
- Which one clearly shows required dependencies?
- Which one allows immutable fields?

Expected conclusion: constructor injection is usually best.

Exercise 4: Interface-Based Loose Coupling

Create an interface:

```
public interface PaymentProcessor {  
    void processPayment(double amount);  
}
```

Then create two implementations:

- `CreditCardPaymentProcessor`
- `PaypalPaymentProcessor`

Inject `PaymentProcessor` into `CheckoutService` .

Use `@Primary` or `@Qualifier` to choose which implementation Spring should inject.

Goal: learn how Spring handles multiple beans of the same interface.

Exercise 5: Use `@Qualifier`

Create:

- `EmailNotificationService`
- `SmsNotificationService`

Both should implement:

```
public interface NotificationService {  
    void sendNotification(String message);  
}
```

Then inject a specific one:

```

public AlertService(@Qualifier("emailNotificationService") NotificationService
notificationService) {
    this.notificationService = notificationService;
}

```

Goal: solve the "multiple matching beans" problem.

Exercise 6: Manual Bean With @Bean

Create a class without @Component :

```

public class CurrencyFormatter {
    private final String currencyCode;

    public CurrencyFormatter(String currencyCode) {
        this.currencyCode = currencyCode;
    }
}

```

Then register it manually:

```

@Configuration
public class AppConfig {

    @Bean
    public CurrencyFormatter currencyFormatter() {
        return new CurrencyFormatter("USD");
    }
}

```

Goal: understand when to use @Bean instead of component scanning.

Exercise 7: Bean Lifecycle

Create a bean with @PostConstruct and @PreDestroy :

```

@Component
public class StartupLogger {

    @PostConstruct
    public void init() {
        System.out.println("Bean initialized");
    }

    @PreDestroy
    public void shutdown() {
        System.out.println("Bean destroyed");
    }
}

```


Goal: observe when Spring initializes and destroys beans.

Exercise 8: Mini IoC Project

Build a small console-based order system.

Classes:

- `OrderService`
- `InventoryService`
- `PaymentService`
- `NotificationService`
- `OrderRepository`

Flow:

```
Place order
Check inventory
Process payment
Save order
Send notification
```

Rules:

- Use `@Service` for business classes.
- Use `@Repository` for storage simulation.
- Use constructor injection everywhere.
- Depend on interfaces where it makes sense.
- Do not use `new` inside service classes.

This is the best full practice exercise for Module 22.

Exercise 9: Test Dependency Injection

Write a unit test for `OrderService`.

Instead of using the real dependency, pass a fake or mock dependency manually.

Goal: see why constructor injection makes testing easier.

Exercise 10: Break It On Purpose

Try these mistakes and observe the errors:

- Remove `@Service` from a dependency.
- Create two implementations of one interface without `@Primary` or `@Qualifier`.
- Move a component outside the scanned package.
- Add a constructor dependency for a class that is not a bean.

Goal: learn Spring errors by experiencing them.

Lab: Spring IoC and Dependency Injection

Build a small Spring Boot console app called **OrderFlow**.

Objective

Practice:

- Spring beans
- constructor injection
- interfaces
- `@Service` , `@Repository` , `@Configuration`
- `@Qualifier`
- bean lifecycle
- loose coupling through IoC

Part 1: Create The Project

Create a Spring Boot project with:

- Spring Web
- Spring Boot DevTools

Package name:

```
com.example.orderflow
```

You can use Spring Initializr or your IDE.

Part 2: Create The Domain Model

```
package com.example.orderflow.model;

public class Order {
    private String itemName;
    private int quantity;
    private double price;

    public Order(String itemName, int quantity, double price) {
        this.itemName = itemName;
        this.quantity = quantity;
        this.price = price;
    }

    public String getItemName() {
        return itemName;
    }

    public int getQuantity() {
        return quantity;
    }

    public double getPrice() {
        return price;
    }
}
```

```
    public double getTotal() {  
        return quantity * price;  
    }  
}
```

Part 3: Create Repository Layer

```
package com.example.orderflow.repository;  
  
import com.example.orderflow.model.Order;  
import org.springframework.stereotype.Repository;  
  
@Repository  
public class OrderRepository {  
  
    public void save(Order order) {  
        System.out.println("Order saved: " + order.getItemName());  
    }  
}
```

Part 4: Create Service Interfaces

```
package com.example.orderflow.service;  
  
import com.example.orderflow.model.Order;  
  
public interface PaymentService {  
    void processPayment(Order order);  
}
```

```
package com.example.orderflow.service;  
  
import com.example.orderflow.model.Order;  
  
public interface NotificationService {  
    void sendNotification(Order order);  
}
```

Part 5: Create Service Implementations

```
package com.example.orderflow.service;  
  
import com.example.orderflow.model.Order;  
import org.springframework.stereotype.Service;  
  
@Service  
public class CreditCardPaymentService implements PaymentService {
```

```
    @Override
    public void processPayment(Order order) {
        System.out.println("Processing credit card payment: $" + order.getTotal());
    }
}
```

```
package com.example.orderflow.service;

import com.example.orderflow.model.Order;
import org.springframework.stereotype.Service;

@Service
public class PaypalPaymentService implements PaymentService {

    @Override
    public void processPayment(Order order) {
        System.out.println("Processing PayPal payment: $" + order.getTotal());
    }
}
```

```
package com.example.orderflow.service;

import com.example.orderflow.model.Order;
import org.springframework.stereotype.Service;

@Service
public class EmailNotificationService implements NotificationService {

    @Override
    public void sendNotification(Order order) {
        System.out.println("Email sent for order: " + order.getItemName());
    }
}
```

Part 6: Create Main Business Service

Use `@Qualifier` because there are two `PaymentService` beans.

```
package com.example.orderflow.service;

import com.example.orderflow.model.Order;
import com.example.orderflow.repository.OrderRepository;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Service;

@Service
public class OrderService {

    private final OrderRepository orderRepository;
```

```

private final PaymentService paymentService;
private final NotificationService notificationService;

public OrderService(
    OrderRepository orderRepository,
    @Qualifier("creditCardPaymentService") PaymentService paymentService,
    NotificationService notificationService) {
    this.orderRepository = orderRepository;
    this.paymentService = paymentService;
    this.notificationService = notificationService;
}

public void placeOrder(Order order) {
    paymentService.processPayment(order);
    orderRepository.save(order);
    notificationService.sendNotification(order);
}
}

```

Part 7: Run Code On Startup

Update your main application class:

```

package com.example.orderflow;

import com.example.orderflow.model.Order;
import com.example.orderflow.service.OrderService;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class OrderFlowApplication {

    public static void main(String[] args) {
        SpringApplication.run(OrderFlowApplication.class, args);
    }

    @Bean
    public CommandLineRunner run(OrderService orderService) {
        return args -> {
            Order order = new Order("Laptop", 1, 1200.00);
            orderService.placeOrder(order);
        };
    }
}

```

Expected output:

```
Processing credit card payment: $1200.0
Order saved: Laptop
Email sent for order: Laptop
```

Part 8: Add Bean Lifecycle

Create:

```
package com.example.orderflow.service;

import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
import org.springframework.stereotype.Component;

@Component
public class ApplicationLifecycleLogger {

    @PostConstruct
    public void init() {
        System.out.println("Spring bean initialized");
    }

    @PreDestroy
    public void cleanup() {
        System.out.println("Spring bean destroyed");
    }
}
```

Part 9: Practice Tasks

After the lab works, try these changes:

1. Change payment from credit card to PayPal using `@Qualifier` .
2. Add `SmsNotificationService` .
3. Use `@Primary` instead of `@Qualifier` .
4. Remove `@Service` from `CreditCardPaymentService` and observe the error.
5. Replace `OrderRepository` with an interface and implementation.
6. Add a second order in `CommandLineRunner` .
7. Write a unit test for `OrderService` using fake dependencies.

Completion Criteria

You are done when you can explain:

- What is a Spring bean?
- What does the application context do?
- Why does `OrderService` not use `new` ?
- Why is constructor injection preferred?
- Why was `@Qualifier` needed?
- How does IoC make the app easier to change?